

Skill &amp; Product Library / Observability

Platform

Design phase

# n8n Execution Intelligence

Point it at any n8n workflow — get root cause, fix suggestion, and a live health dashboard in under a minute.



Target: failure diagnosis from 10-15 min to under 60 seconds

n8n

Python

SQLite

observability

SRE

## Overview

An observability layer for your n8n automation stack. Paste a workflow ID or n8n link and NEI ingests the entire execution history via the n8n Public API (GET-only by charter), archives every run — full raw JSON included — into a master `.md` knowledge file, and runs a three-layer analysis: failures + fixes, performance, and health. Ask "why did this execution fail?" and get a plain-English root cause with a suggested fix, citing real execution IDs. An auto-updating HTML dashboard gives the whole team at-a-glance automation health. Target: cut failure diagnosis from 10–15 minutes of clicking to under 60 seconds.

## The Problem

At a global HR-tech (EOR) platform (~500 employees, operating in 160+ countries), the prospect pipeline runs on n8n automations chaining intent data → HubSpot → n8n → Clay → sequencing → dialer. When a workflow failed or quietly stalled, the team found out only when prospect data stopped moving — then diagnosed it by clicking through executions in the n8n UI one at a time, 10–15 minutes per incident, with no shared view of automation health.

## How It Works

1. **Front door** — a Claude skill/agent (or CLI) accepts a workflow ID or an n8n URL and resolves the target on the company's n8n Cloud instance.
2. **Ingest (read-only)** — a Python client pulls every execution via the n8n Public API v1 with cursor pagination. Plain English: it downloads the workflow's full run history without ever being able to change anything.
3. **Parse + store** — each execution's status, timing, per-node `runData`, failing node, and error message land in SQLite (Postgres-ready) plus a per-workflow master `.md` archive with the raw JSON. Idempotent upsert: re-runs update, never duplicate.
4. **Three-layer analysis** — (a) failures + fixes with a 9-category error taxonomy (auth, rate\_limit, timeout, mapping\_error, downstream\_http, data\_shape, manual\_stop, waiting\_unresumed, unknown); (b) performance (duration, node timing, p95, success rate); (c) health (stall detection, trends, composite health score).
5. **Grounded answers** — natural-language diagnosis where every number traces to a real execution ID. Absent data = null; no fabrication, ever.
6. **Dashboard + alerts** — a self-contained HTML/Chart.js dashboard (shareable with the go-to-market team and leadership) refreshes on each run; alerting ships dry-run by default. Optional v1.1: correlate against HubSpot/Clay to confirm records actually landed.



## Stack & Integrations

| Tool                                | Role                          | Access needed                  |
|-------------------------------------|-------------------------------|--------------------------------|
| n8n Public API v1                   | Execution/workflow ingestion  | API key in env, GET-only       |
| Python 3 (requests, pandas, jinja2) | ETL + analysis pipeline       | Local/sandboxed runtime        |
| SQLite                              | Executions + node_runs store  | Local file, Postgres-ready DAL |
| Chart.js / Plotly                   | Dashboard rendering           | None (render only)             |
| Claude skill                        | NL front door + orchestration | Claude Code / Cowork           |
| HubSpot MCP/REST (v1.1)             | Did-records-land correlation  | Least-privilege read scopes    |
| Clay API/webhook (v1.1)             | Enrichment-run correlation    | Table read access              |

## Setup Process

1. Provision an n8n Public API key on your instance and export it as an environment variable (never committed or logged).

2. Install the skill / clone the pipeline; run the API Readiness check (a 401 blocks with a clear message instead of failing silently).
3. Pilot on one workflow with `--limit` to validate the master `.md` schema and error taxonomy on real data.
4. Run full ingestion; verify idempotent upsert by re-running.
5. Generate the dashboard; share the HTML artifact or Cowork live artifact.
6. Enable the scheduled runner (default daily); leave alerts in dry-run until output is trusted.

## What You Need

- [ ] n8n Cloud/self-hosted instance with Public API enabled + API key (read scope)
- [ ] Python 3 runtime (local or sandboxed)
- [ ] Claude Code/Cowork for the skill front door (optional — CLI works standalone)
- [ ] Optional v1.1: HubSpot read token, Clay table access
- [ ] Time: ~1 hour to deploy once built; full build plan is 16 prioritized tasks, ~41 hours



## Results

- **Targets (design-phase, honestly scoped):** time-to-diagnosis from 10–15 min to **under 60 seconds**; stalled workflows detected **within 24 hours**; **≥80%** of failures shipped with an actionable fix suggestion.
- Working read-only API client, SQLite store, diagnosis engine, and HTML dashboard already shipped in a prior execution-analysis skill — this system formalizes and extends it.
- Full leadership-ready package (brief, PRD, architecture, prioritized build plan, build prompts) generated at 92% package quality via the builder's own project-intake framework.

## Challenges & Fixes

- **Silent failure discovery** → failures surfaced only when data stopped moving → stall detection with SLA windows and a composite health score, checked on a daily schedule.
- **LLM fabrication risk in diagnosis** → analysis could "sound right" without evidence → hard charter: every metric traceable to a real execution ID, absent data rendered as null, alerts dry-run by default.
- **Large execution histories** → full pulls risk rate limits → cursor pagination with backoff + resume; partial archives explicitly flagged.
- **Safety of pointing tools at production** → strict GET-only design: the orchestrating agent never calls a write endpoint; no auto-remediation in v1.

## Demonstrates

SRE instincts applied to business automation · read-only-by-design safety · grounded (no-fabrication) AI analysis · the maturity step from building automations to operating a platform.

### STACK & INTEGRATIONS

n8n

Python

SQLite

Chart.js

HubSpot (optional)

Clay (optional)

### WHAT IT NEEDS

- ✓ n8n Cloud/self-hosted instance with Public API enabled + read-scope API key
- ✓ Python 3 runtime (local or sandboxed)
- ✓ Optional: Claude Code/Cowork for the skill front door



### DISCUSS THIS BUILD

Happy to walk through the architecture, the failures along the way, or how this could translate to your team.

Email Patrick

© 2026 Patrick Diamitani. Case studies anonymized for public portfolio use — no employer-identifying or client-identifying detail included.

[pdiamitani@atlashxm.com](mailto:pdiamitani@atlashxm.com) · [linkedin.com/in/diamitani](https://www.linkedin.com/in/diamitani)